

AI GENERALIST ACCELERATOR PROGRAM

Building a Lovable Interface

Customer Support Triage Bot

A Complete Step-by-Step Guide

USE CASE — Build a customer-facing web form for the Day-4 Customer Support Triage Bot, so real customers can submit feedback from a web page and have it automatically triaged, logged, acknowledged, and escalated.

API KEY: To Be Added

What You Will Do — At a Glance

- **What you build:** A friendly customer-support form (built in Lovable) wired to your Day-4 triage bot in n8n. A customer submits the form → the bot classifies the feedback, logs it to Google Sheets, emails the customer an acknowledgement, and escalates High-urgency issues to your team.
- **What you'll learn:** how a webhook connects a web front-end to n8n, how to keep your webhook URL safe using a secret, and how to publish the whole thing live.
- **Before you start:** you need the Customer Support Triage Bot you built on Day 4 (its workflow + Google Sheet), an n8n account, a Lovable account, an API key, and a test inbox you can check.

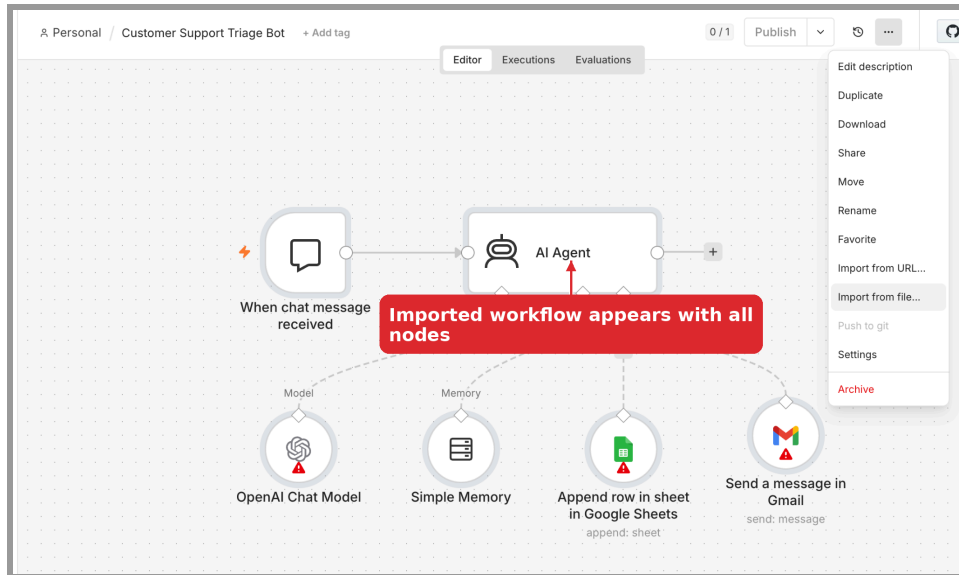
Part A — Prepare Your Triage Bot in n8n

Step 1: Open your triage workflow

- Open the **Customer Support Triage Bot** workflow you built on Day 4. You'll modify it in Part C.



No Day-4 workflow?: Import the JSON: [Customer Support Triage Bot Json](#) on a new n8n canvas, open the ⋮ menu → **Import from File**, or paste the copied JSON straight onto the canvas. All nodes appear automatically.



Step 2: Check your Google Sheet

- Open your **Customer Feedback Tracker** sheet (or make a copy of the sample from Day 4).
- Row 1 must have these 7 headers, in order: **Date | Customer Name | Customer Email | Feedback Summary | Category | Urgency | Status**

The screenshot shows the Google Sheets interface for a sheet named 'Customer Feedback Tracker'. A red callout box with the text 'Add these 7 column headers in Row 1' points to the first row of the table. The table has 7 columns and 4 rows. The first row contains the headers: Date, Customer Name, Customer Email, Feedback Summary, Category, Urgency, and Status. The second row is empty, and the third and fourth rows are also empty.

	A	B	C	D	E	F	G
1	Date	Customer Name	Customer Email	Feedback Summary	Category	Urgency	Status
2							
3							
4							

Step 3: Confirm your credentials

The workflow needs three connections. Open each node and confirm:

- **OpenAI Chat Model** — your API key credential is added, and Model = **gpt-4.1-mini**.

Parameters
Settings

Credential

☐ n8n Connect No API key required

☒ My own credential Use your own API key Add your API key

OpenAI account

Model

From list gpt-4.1-mini Model: gpt-4.1-mini

☐ Use Responses API

Options

+ Add Option

? I wish this node would...

- Google Sheets Tool** — credential added, Operation = **Append Row**, Document = Customer Feedback Tracker, Sheet = Sheet1, and every column left as **Filled by AI**.

Parameters
Settings
Execute step

Credential

Google Sheets OAuth2 API

Tool Description

Set Automatically

Resource

Sheet Within Document

Operation

Append Operation: Append Row in Sheet

Document

From list Customer Feedback T...

Sheet

From list Sheet1

Mapping Column Mode

Map Each Column Manually

Parameters
Settings
Execute step

Date

Defined automatically by the model

+ Add a description

Customer

Every column: Filled by AI

Defined automatically by the model

+ Add a description

Customer Email

Defined automatically by the model

+ Add a description

Feedback Summary

Defined automatically by the model

+ Add a description

Category

Defined automatically by the model

+ Add a description

Urgency

Defined automatically by the model

+ Add a description

Status

Defined automatically by the model

+ Add a description

- **Gmail Tool** — a Gmail account is connected, Resource = Message, Operation = Send, with To / Subject / Message set to **Filled by AI**.

Step 4: The triage prompt (already set)

The agent's behaviour is already configured in its **System Message** — it reads each piece of feedback and extracts Name, Email, a one-line Summary, a Category (Bug / Feature Request / Complaint / Question / Praise) and an Urgency (High / Medium / Low). It then logs a row to the Sheet, emails the customer an acknowledgement, and — only when Urgency = High — emails an escalation to your team.

✓ **One thing to change:** Open the AI Agent's System Message and replace **team@yourcompany.com** with your real team address. For testing, use your own email so you can confirm escalations arrive.

Part B — Build the Interface in Lovable

Step 5: Generate your support form

- Open [Lovable](#) → Sign in → Click on **New Project**.
- Paste the prompt below and press Enter. It names the exact fields and tells Lovable to read the webhook URL from a secret — which makes the n8n hookup easy later.

Build a clean, modern customer support feedback form as a single page.

The form should have:

- A heading "Customer Support" with a short friendly subtitle
- A text field: "Your Name"
- An email field: "Your Email"

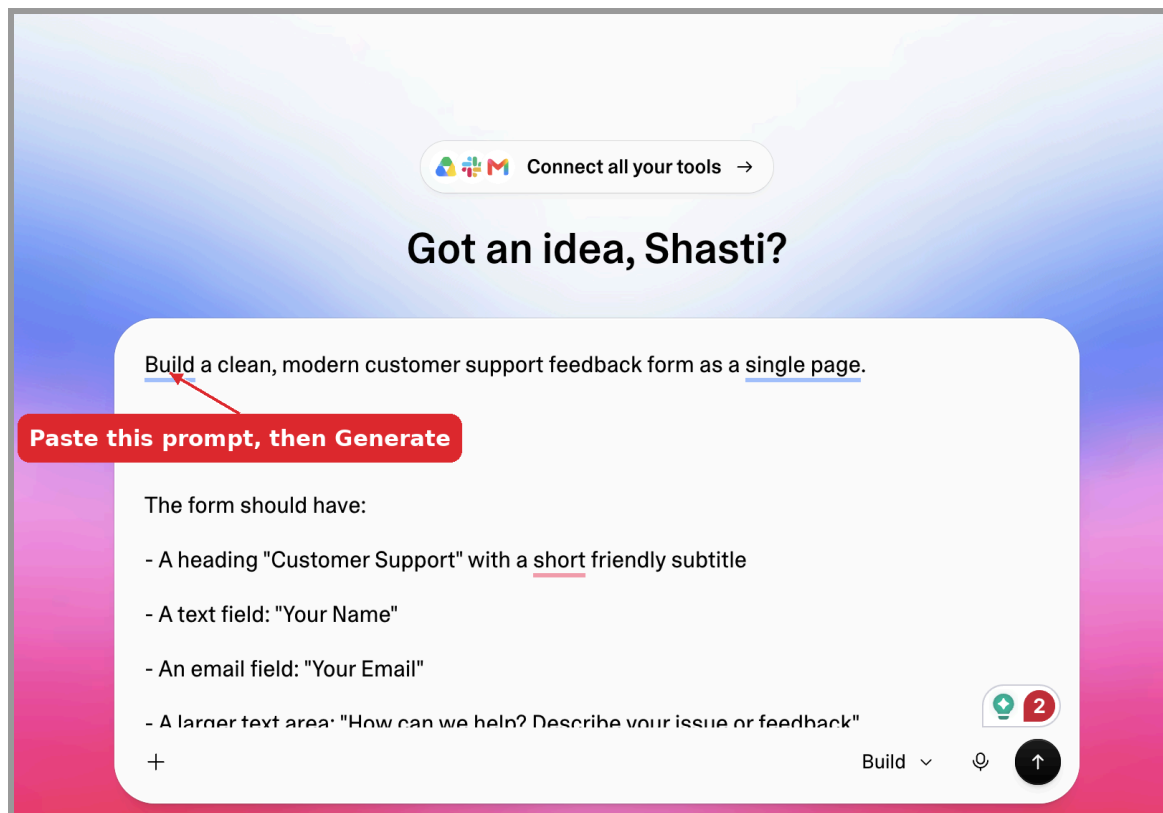
- A larger text area: "How can we help? Describe your issue or feedback"
- A "Submit" button

When the user clicks Submit:

- Send the form data as a JSON POST request to a webhook.
- Read the webhook URL from a secret named `N8N_WEBHOOK_URL`. Do NOT hardcode it.
- The JSON body must use exactly these keys: `customerName`, `customerEmail`, `feedback`
- Show a loading state, then on success show a thank-you message.

No backend or database is needed. Keep it friendly with soft colours and rounded corners.

- Wait for Lovable to build the page, then check it has **Your Name**, **Your Email**, a message box, and a **Submit** button.



i Keep the keys exact: The three JSON keys — **customerName**, **customerEmail**, **feedback** — are what n8n will receive. Keeping them exactly as written makes the mapping in Part C straightforward.

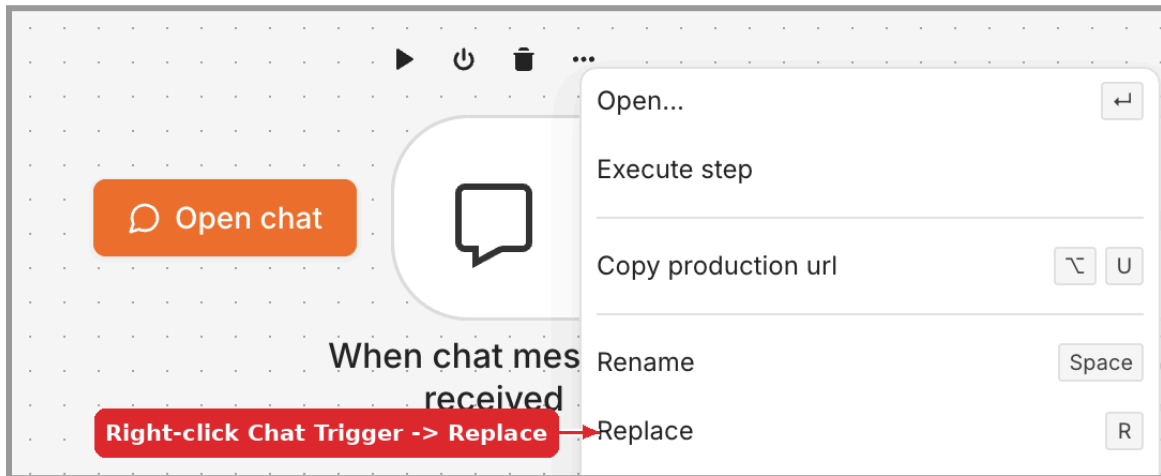
Part C — Connect the Form to the Bot with a Webhook

A **webhook** is a URL that works like a doorbell: when a customer submits the form, Lovable “rings” the webhook, your n8n workflow runs, and it sends a reply back so the customer sees a success message.

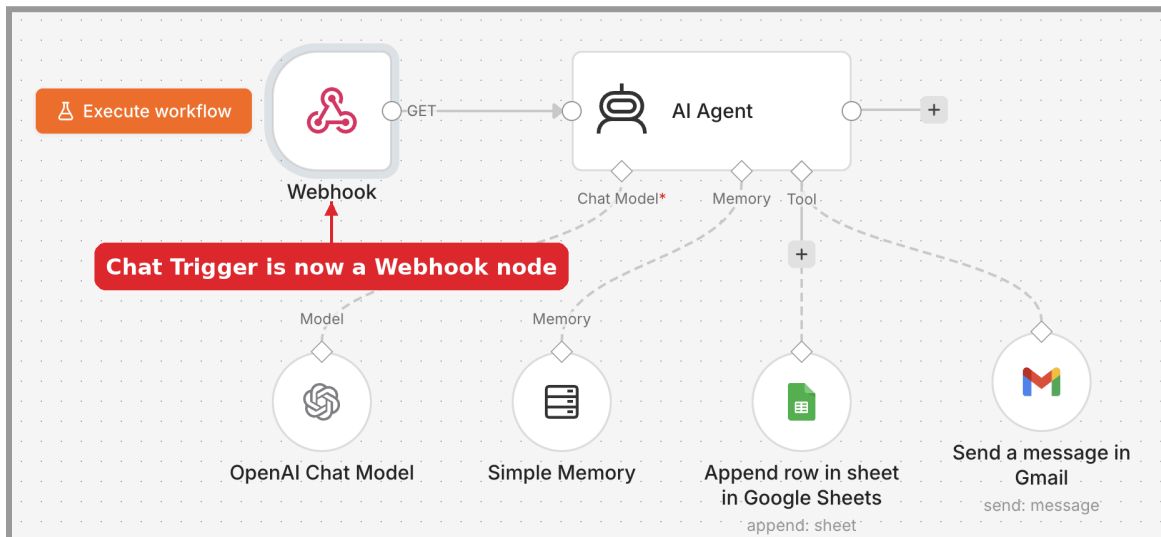
Step 6: Replace the Chat Trigger with a Webhook

Node type: Webhook

- In your triage workflow, **right-click the Chat Trigger** node → **Replace** → search for the trigger “**Webhook**”.



- The trigger is now a **Webhook** node feeding your AI Agent.



Step 7: Configure the Webhook

- Click the Webhook node. Set **HTTP Method** → **POST**.
- Set **Respond** → Using ‘**Respond to Webhook**’ Node.
- Leave Path as its default.

Parameters Settings Listen for test event

Webhook URLs

Test URL Production URL

POST https://outskill-gef.app.n8n.cloud/webhook-test/c3e1b2ed-85dd-49dc-aec7-a888d1ce74cf

HTTP Method

POST HTTP Method -> POST

Path

c3e1b2ed-85dd-49dc-aec7-a888d1ce74cf

Authentication

None

Respond

Using Respond -> Using 'Respond to Webhook' Node

Insert a 'Respond to Webhook' node to control when and how you respond. [More details](#)

Options

+ Add option

! **Important:** The method must be **POST**, or the form's data won't come through.

Step 8: Copy the Production URL

- In the Webhook node, open **Webhook URLs** and switch to the **Production URL** tab.
- Click **Copy** — you'll paste it into Lovable in Step 10.

Parameters Settings Listen for test event

Webhook URLs

Test URL Production URL

Switch to Production URL and Copy

POST https://outskill-gef.app.n8n.cloud/webhook/c3e1b2ed-85dd-49dc-aec7-a888d1ce74cf

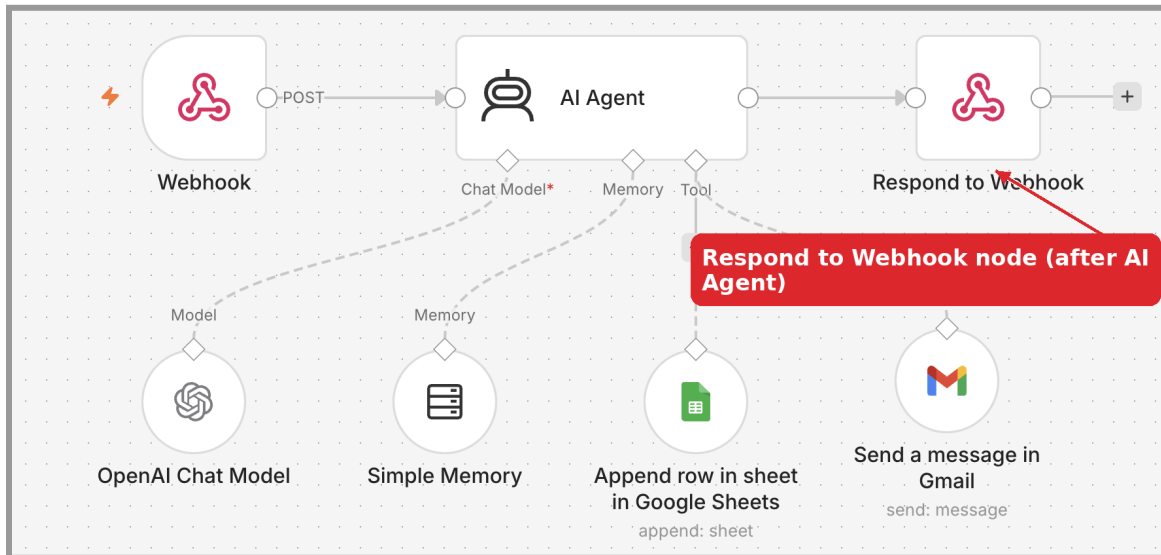
! **Warning:** Use the **Production URL**, not the Test URL. The Test URL only works while you click “Listen for test event” in n8n.

Step 9: Add a Respond to Webhook node

Node type: Respond to Webhook

Purpose: sends a reply back to Lovable so the customer sees the success message.

- Click the + after the **AI Agent** and add **Respond to Webhook**.



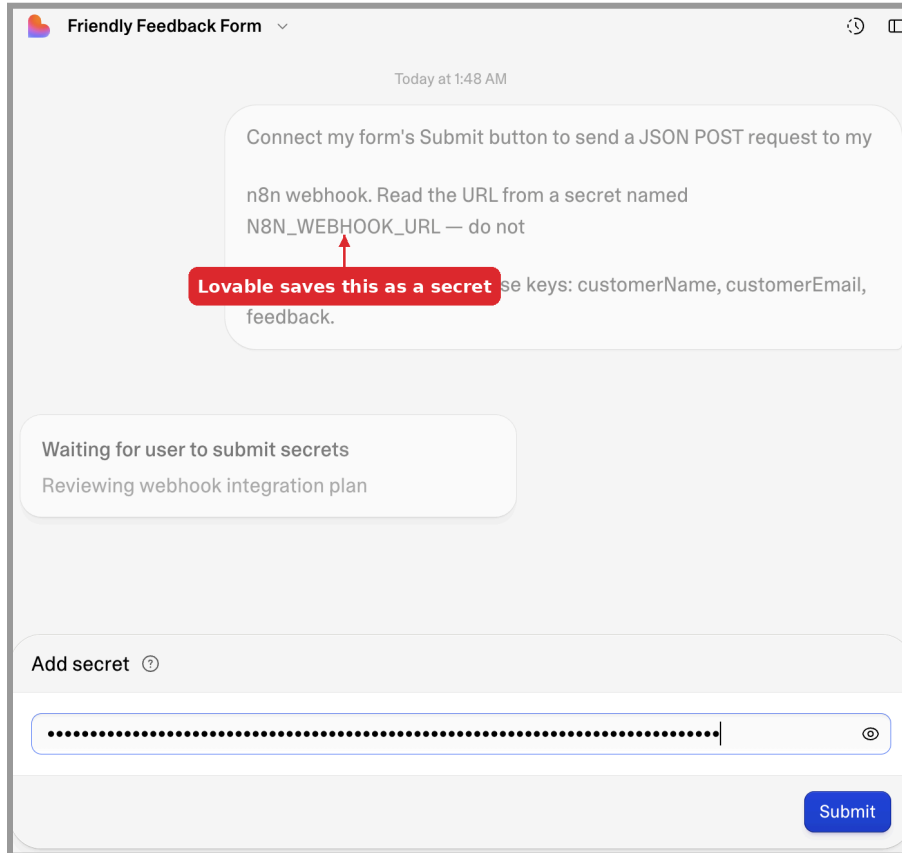
- Open it and set **Respond With** → **All Incoming Items**. Then Save.

The screenshot shows the configuration interface for the 'Respond to Webhook' node. At the top, there are tabs for 'Parameters' and 'Settings', and an 'Execute step' button. A warning message states: 'Verify that the "Webhook" node's "Respond" parameter is set to "Using Respond to Webhook Node". [More details](#)'. Below this, the 'Respond With' dropdown is set to 'All Incoming Items'. A red callout box with an arrow pointing to the dropdown contains the text 'Respond With -> All Incoming Items'. There is also an 'Options' section with a '+ Add option' button.

Step 10: Store the URL as a secret in Lovable

Your webhook URL is private, so store it as a **secret** instead of pasting it into the code.

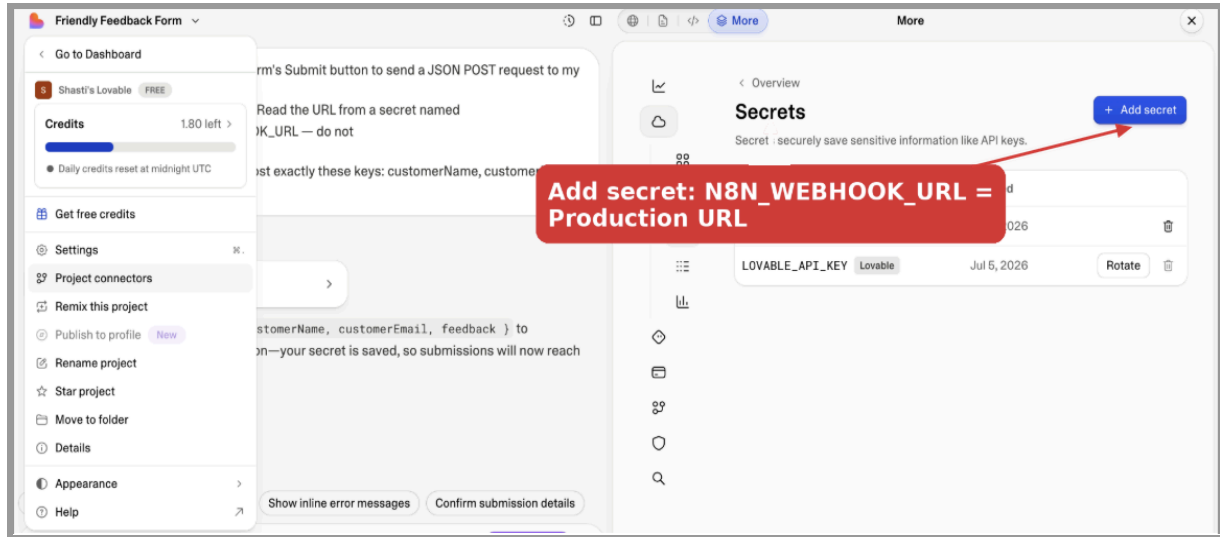
- In Lovable's chat, you can reinforce the wiring (optional): *“Connect my form's Submit button to POST to my n8n webhook, reading the URL from a secret named N8N_WEBHOOK_URL. Post keys: customerName, customerEmail, feedback.”*



The screenshot shows the Lovable chat interface. At the top, it says "Friendly Feedback Form". Below that, a chat message from "Today at 1:48 AM" reads: "Connect my form's Submit button to send a JSON POST request to my n8n webhook. Read the URL from a secret named N8N_WEBHOOK_URL — do not use keys: customerName, customerEmail, feedback." A red callout box with an arrow pointing to the text "N8N_WEBHOOK_URL" contains the text "Lovable saves this as a secret". Below the chat message, there is a status bar that says "Waiting for user to submit secrets" and "Reviewing webhook integration plan". At the bottom, there is a section titled "Add secret" with a question mark icon. Below this title is a text input field filled with dots, and a "Submit" button is located to the right of the input field.

Alternative Step:

- Open your Lovable project **Settings** → **Secrets** (Project Connector → Cloud → Secrets). Click on **Add Secret** and add your **N8N_WEBHOOK_URL** and paste your **Production URL** as the value. Save.



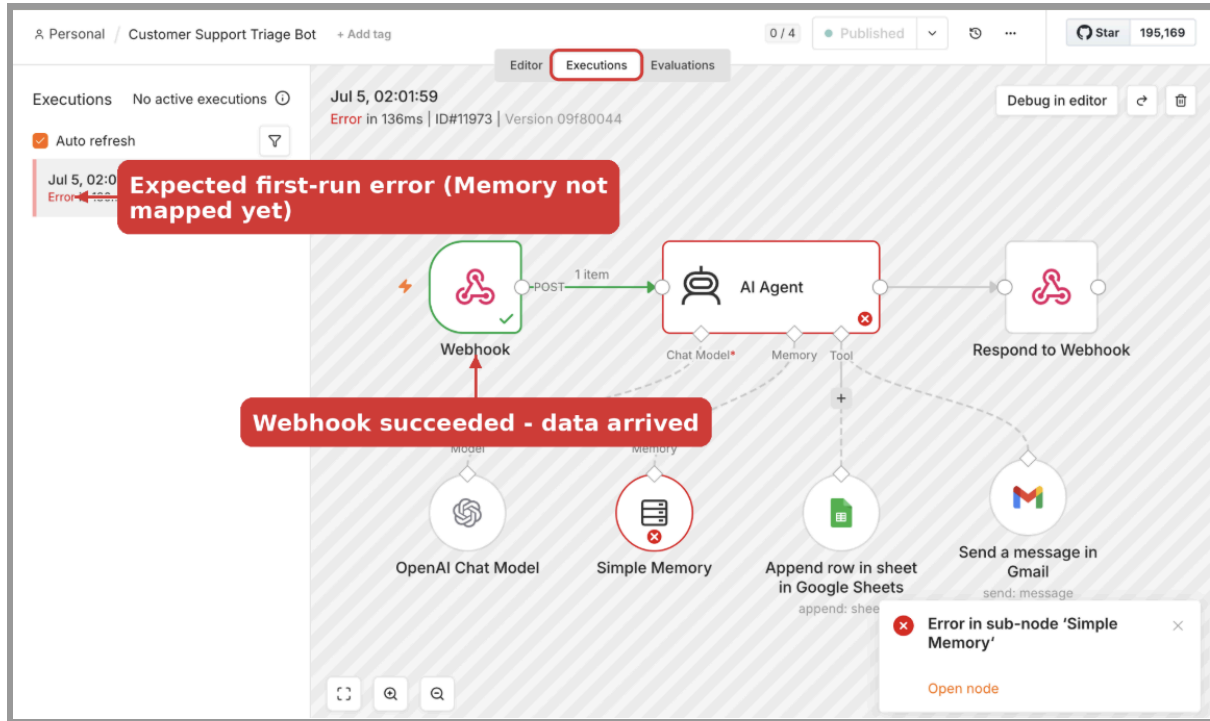
⚠ **Warning:** Never paste the webhook URL into public chat or committed code — keep it only in the secret.

💡 **Tip:** Can't find the Secrets panel? Ask Lovable directly: “Where do I add a secret for this project?”

Step 11: Map the webhook data into your nodes

The nodes still expect input from the old Chat Trigger. Point them at the Webhook instead — no coding, just drag fields. Send one test submission first so n8n can see the data.

- **Send a test ticket** from your Lovable form's preview, e.g. Priya / priya@acme.com / “Export feature broken for 2 days, we're losing clients, fix this NOW”.
- In n8n, open **Executions**. The first run shows an **Error on Simple Memory** — this is expected, because the nodes still point at the old trigger. Note the **Webhook node ran green**, so the data arrived.



- **Simple Memory node:** set Session ID → **Define below**; Key → a static value **triage-session**.

The screenshot shows the configuration for the Simple Memory node. The 'Parameters' tab is active, displaying the following settings:

- Session ID:** Set to 'Define below' (indicated by a red annotation 'Session ID -> Define below').
- Key:** Set to 'triage-session' (indicated by a red annotation 'Key -> triage-session').
- Context Window Length:** Set to 5.

A descriptive text box explains that this node stores memory locally in the n8n instance and is not compatible with Queue Mode or Multi-Main setups.

- **AI Agent node:** set Source for Prompt (User Message) → **Define below**; in the Prompt box,

Option 1: Copy the below prompts

New customer support ticket received.

```
Customer Name: {{ $json.body.customerName }}  
Customer Email: {{ $json.body.customerEmail }}  
Feedback: {{ $json.body.feedback }}
```

Parameters

Settings

Execute step

Tip: Get a feel for agents with our quick [tutorial](#) or see an [example](#) of how this node works

Source for Prompt (User Message)
Define

Source for Prompt -> Define below

Prompt (User Message)

fx

New customer received.
Customer Name: {{ \$json.body.customerName }}
Customer Email: {{ \$json.body.customerEmail }}
Feedback: {{ \$json.body.feedback }}

Map the webhook fields here

Require Specific Output Format

Enable Fallback Model

Options

Option 2: Drag and Drop the Webhook Fields.

OPTION 2 · Drag & drop

Drag any of these fields into the box

body

customerName Bharadwaj Y.S.N

customerEmail programs@outskill.com

feedback Your website is very cool

Source for Prompt (User Message)
Define

Source for Prompt -> Define below

Prompt (User Message)

fx

New customer received.
Customer Name: {{ \$json.body.customerName }}
Customer Email: {{ \$json.body.customerEmail }}
Feedback: {{ \$json.body.feedback }}

Map the webhook fields here

- **Re-submit** the ticket from Lovable — this run should now complete green.

⚠ Troubleshooting: If Simple Memory still errors, make sure the Key is a static value (**triage-session**), not an expression.

Part D — Test and Go Live

Step 12: Run the end-to-end test

Submit these three tickets from your form, one at a time, waiting for the success message after each:

- **Priya** / priya@acme.com / “Export feature broken for 2 days, we’re losing clients, fix this NOW” → Bug, **High** → row + acknowledgement + **team escalation**.
- **Rohan** / rohan@startup.in / “Loving the new dashboard, would love a dark-mode option” → Feature Request, Low → row + acknowledgement only.
- **Anjali** / anjali@retail.com / “How do I invite team members?” → Question, Low → row + acknowledgement only.

Then verify:

- **Google Sheet:** a new row per ticket, with the right Category, Urgency, Date, and Status = New.

	A	B	C	D	E	F	G	H	I
1	Date	Customer Name	Customer Email	Feedback Summary	Category	Urgency	Status		
2	2024-06-10	Priya	priya@acme.com	Export feature broken for 2	Bug	High	New		
3	2024-06-10	Rohan	rohan@startup.in	Loves new da					
4	2024-06-10	Anjali	anjali@retail.com	Question abo					

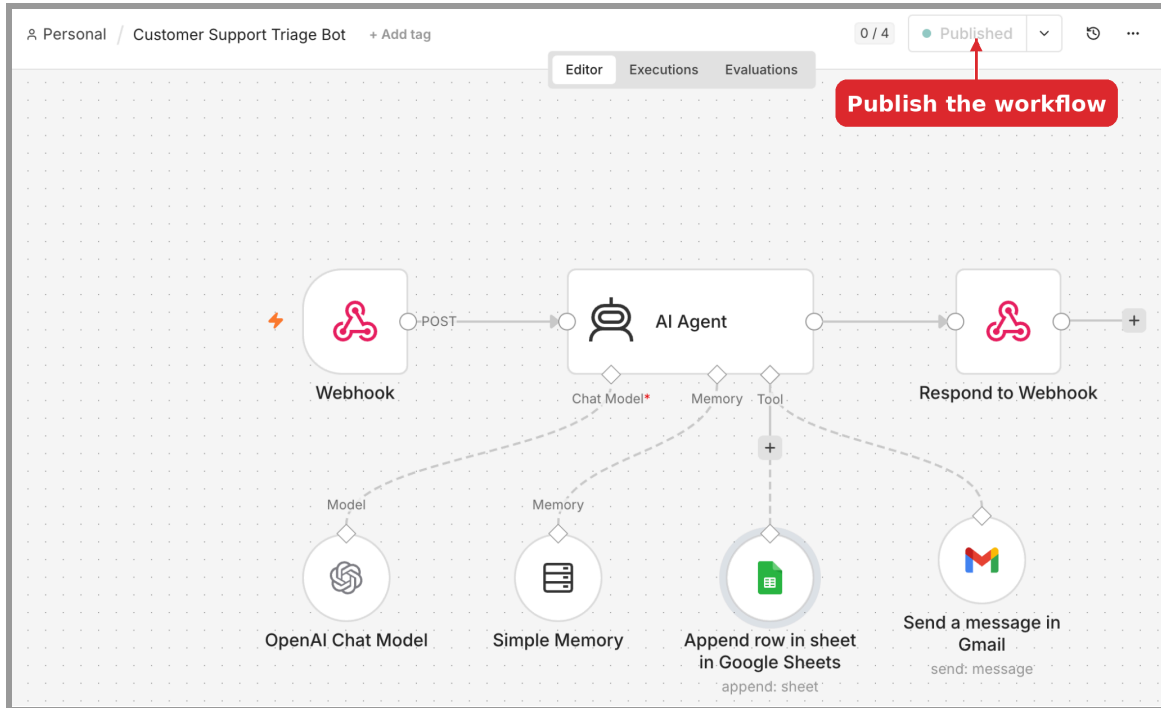
AI-classified Categ Status = New

- **Customer inbox:** a warm acknowledgement for each ticket.
- **Team inbox:** an escalation for **Priya only** (not Rohan or Anjali).

Step 13: Publish and go live

Three things make it live for real customers:

- **Publish the n8n workflow** (toggle to Active / Published). It now listens on the Production URL automatically — no more “Listen for test event”.



- **Publish your Lovable app** so the live site uses the secret. Note your live URL.

Screenshot of the Lovable app interface showing the 'Published' status and a 'Publish your app' button. The interface includes a form for user registration with fields for Name, Email, and a feedback text area, and a 'Submit' button. A red arrow points to the 'Published' status.

- **Final test:** open your published form as a customer would and submit a ticket — it should run automatically and log + email exactly as before.

Customer Support

We're here to help. Tell us what's on your mind and we'll get back to you.

Your Name

Your Email

How can we help?

Submit

Common Issues & Fixes

- **“Webhook not found”** → use the Production URL (not Test); the workflow must be Published; check the URL was copied fully into the secret.
- **Form submits but nothing runs** → the Webhook method must be POST; the JSON keys must be `customerName` / `customerEmail` / `feedback`.
- **Agent runs but fields are empty** → re-drag the `{{ $json.body.* }}` fields; send a fresh test ticket first so the output is current.
- **Simple Memory error** → set the Session Key to a static value (`triage-session`).
- **No row / no email** → re-check the Google Sheets and Gmail credentials and that the right sheet/tab is selected.

Conclusion

Congratulations! You've connected a customer-facing Lovable form to your n8n triage bot. Customers can now submit feedback from a web page, and behind the scenes it's automatically classified, logged to Google Sheets, acknowledged by email, and escalated to your team when it's urgent.

This exact pattern — **Lovable form** → **webhook** → **n8n** → **response** — powers almost any automation you can imagine.